LANFORGE × MODEL CONTEXT PROTOCOL

Observations and Proposals

Macha Praveen — April 2026

Thesis

> *LANforge already exposes a self-describing JSON-over-HTTP API (`/cli-json/:cmd`, `/port/:resource/:port`, `/events`, `/help/:cmd`). Structurally, this is an MCP target waiting to happen — a test harness an LLM can drive directly without a single line of glue per command. Spirent Octobox cannot follow because its automation surface is GUI-locked. LANforge's decade-old CLI-over-HTTP architecture is the moat.*

What lanforge-mcp ships today (v0.1.0)

- **Six MCP tools** wrapping the most-used LANforge actions: `list_ports`, `create_stations`, `start_l3_traffic`, `get_test_results`, `get_events`, `diagnose_failure`. Mutating tools default to `dry_run=True`; the LLM must explicitly set `dry_run=False` to execute, and dry-run returns the exact CLI-JSON payload that would have been POSTed.
- **Mock-first development** — every tool works end-to-end against recorded JSON fixtures (sourced from the candelatech.com cookbook); `LANFORGE_MOCK=1` is the default, so the demo runs offline, in CI, and on a laptop without any LANforge appliance. A single env-var flip points at a real GUI on `localhost:8080`.
- **Engineering rigour** — Python 3.10+, `mypy --strict` clean, `ruff` clean, 63 tests / 88% coverage, rule-based diagnostic summaries (no LLM in the diagnostic path), single-page interactive demo on GitHub Pages.

What 90 days of focused work could add

- **Real-hardware verification + flag-bit table** against the Candela demo lab (no schema changes expected; v0.1's TODOs around unverified `add_sta` flag bits become concrete defaults).
- **TR-398 issue 4 prompt library**: capacity, range, roaming, multi-station latency, all expressible as parameterised MCP prompts the LLM expands into concrete plans against `list_ports` → `create_stations` → `start_l3_traffic` → `diagnose_failure`.
- **Chamber View / attenuator coverage** as additional read/write tools (turntable position, RF noise floor, path-loss profile) — the existing tool taxonomy extends naturally without rearchitecture.

Repository: `github.com/machapraveen/lanforge-mcp` · Live demo: `{{PAGES_URL}}`

Macha Praveen | `github.com/machapraveen` | `praveenmacha.p1@gmail.com`

CODE QUALITY

Observations on greearb/lanforge-scripts

These are surface-level, low-cost-to-fix issues I noticed while reading the public mirror of `github.com/greearb/lanforge-scripts`. None of them are blockers; collectively they are a half-day of tightening that pays off in PR-friendliness, downstream packaging, and developer onboarding.

Toolchain consistency

- Inconsistent shebangs across entry scripts — `#!/usr/bin/env python3` in some files, `#!/usr/bin/python3` in others. The `env` form is the portable choice and matches the way most distros ship today.
- Runtime `sys.version_info` checks in scripts (e.g. enforcing 3.6+) belong in `python_requires` on a `pyproject.toml` or `setup.cfg` instead. Right now there is neither a `pyproject.toml` nor a `setup.py` at the repo root, which blocks `pip install` against the repo and forces the documented `sys.path.append('../../../../')` pattern in entry scripts.
- `# flake8: noqa` file-wide disables in `ap_ctl.py` and a few other modules. Inline `# noqa: <rule>` with justifying comments is friendlier to PR review (and to anyone who runs `ruff`/`flake8` as a smoke test).
- No CI workflow detected, no Dockerfile, no `tox.ini`. A 30-line GitHub Actions matrix that just runs the existing test scripts under Python 3.9–3.12 would be high-leverage.

Secrets and credentials

- Hard-coded credentials in `wifi_ctl_9800_3504.py` (username `cisco` / password `Cisco123` / management IPs in the `172.19.27.x` block). These almost certainly correspond to a lab-internal AP, not a production device, and are fine for a dev-environment CLI helper. But they appear on first `grep` for anyone auditing the repo — moving them to `os.environ.get('WIFI_CTL_USER')` with documented env-var defaults makes the file safe to point a security tool at.

Architecture seams

- Dynamic `LF_USE_AUTOGEN` import in a couple of script entry points suggests an in-flight autogen-based client that hasn't fully landed. Either committing to the autogen path or removing the conditional reduces surprise for new contributors trying to follow the import graph.
- Several scripts open with `sys.path.append('../../../../')` to find sibling modules. A `pyproject.toml` with a `src/` layout fixes this once and for all — every script becomes runnable from any cwd.

I have not opened a PR against `greearb/lanforge-scripts`. These notes are diagnostic only; happy to take this list in priority order on day one.

DOCUMENTATION & UX

Surface gaps on candelatech.com

Same framing as the previous page: nothing here is a blocker. These are friction points I hit while building `lanforge-mcp` without access to the appliance, where the public web is the only source of truth.

Discoverability

- `forum.candelatech.com` does not resolve. The community discussion that does exist seems to live on `forum.openwrt.org` tagged `lanforge`. A canonical forum (or a GitHub Discussions tab on `greearb/lanforge-scripts`) would consolidate it.
- `candelatech.com` has no site search. For a docs-heavy domain (~hundreds of cookbook pages, the LFCLI user guide, the JSON API ref), this is the highest-leverage UX fix on the list. A static `pagefind`/`lunr` index over the existing HTML would do it in a day.
- The blog lives at `lanforge.wordpress.com`, disconnected from the main domain. Folding it into `candelatech.com/blog` (or just linking it from the top nav) keeps Google's link graph consolidated.

Reference quality

- The JSON API is described prose-by-example in `candelatech.com/cookbook.php?vol=cli&book=JSON__Querying_the_LANforge_Client_for_JSON_Data`. There is no machine-readable schema. An OpenAPI 3.1 spec generated from the existing `/help/:cmd` endpoint would give us auto-generated SDKs for every language, plus this MCP server's tool definitions for free.
- The cookbook itself acknowledges that *"clearly, the JSON output is difficult to read"*. The fix is upstream: returning `{ "port": <name>, "alias": <...>, ... }` with proper snake_case keys instead of `{ "1.5.wiphy0": { "tx bytes": 0, ... } }`. I understand the historical reason (eid-as-key), but a `?format=v2` query parameter unblocks every modern client without breaking existing scripts.
- Several install pages still reference Fedora-24-era instructions (GPG keys, RPM URLs). A one-paragraph *"supported on Fedora 38+ and Rocky 9 today"* note plus a current-state install command saves prospective evaluators a confusing afternoon.

Marketing surface

- The Candela YouTube channel has no Wi-Fi 7 / 802.11be deep-dive content as of this writing. The competitive narrative against Spirent right now is that LANforge tracks the standard at the kernel level and Spirent doesn't — but you have to know that to find it. A 10-minute walk-through video where Ben sets up an MLO test on real hardware would close the gap.

PROPOSAL

First 90 days

If hired into a Wi-Fi automation engineering role at Candela, this is what I'd want to ship in three months. Numbers are deliberate goals, not promises — they exist so my work is auditable.

Days 1–15 — Embed and ground-truth

Run the 20 most common test scenarios on real hardware end-to-end. Shadow inbound support tickets to find the actual user pain (vs. what I imagined from the public docs). Identify which `add_sta` / `add_cx` flag bits are the v0.1 TODOs in `lanforge-mcp` and finalize them against a Candela demo lab. Goal: zero `TODO` comments in `tools.py` by day 15.

Days 16–45 — Ship lanforge-mcp v1

Cut `v1.0` against verified hardware: real flag bits, real edge-case event severities, an OpenAPI 3.1 spec generated from the existing `/help/:cmd` endpoint, and an MCP-Inspector-screenshot test in CI. In parallel, do a type-hint pass on `py-json/` in `greearb/lanforge-scripts` behind a feature branch — small surface, high downstream value (every IDE now knows what each LANforge response shape looks like).

Days 46–90 — Open-source and pitch

Move `lanforge-mcp` to the Candela GitHub org with appropriate naming and a Candela-branded release. Publish a case-study post on `candelatech.com/blog` (not WordPress) with the demo URL — "How we shipped MCP support in 90 days" becomes a credible recruiting story for the next hire. Use the remaining cycles to scope two follow-ons: an agentic regression bot that runs the TR-398 prompt library nightly against the demo lab and posts a Slack/email digest, and an AI-test-report Test-as-a-Service tier that customers can plug their LANforge into.

Why I'm credible to do this

`lanforge-mcp` v0.1 is the artifact of this proposal: built in 48 hours, fully type-checked, 88% test coverage, mock-first so it runs without your hardware, and interactive on the web for any reviewer to click through. The code is the resume.

Macha Praveen | github.com/machapraveen | praveenmacha.p1@gmail.com